

# **Software Requirements Specification**

*for*

## **Library Management System**

*(Python and Database based application)*

Version 1.0

Prepared for academic submission

## Table of Contents

# 1. Introduction

## 1.1 Purpose

This document describes the software requirements for a Library Management System (LMS). The purpose of the system is to automate the day-to-day activities of a library, such as adding books, registering members, and issuing or returning books, using a Python application connected to a database. This document is intended for the developer building the project and for evaluators reviewing it as part of academic coursework.

## 1.2 Scope

The Library Management System will allow a librarian to manage the library's book catalogue and member records, and to track which books are issued to which members. The system will be a single-user, menu-driven desktop application built in Python, using SQLite as the database. It will not include online access, a web interface, or payment gateway integration; fine calculation will be limited to a simple per-day rate.

## 1.3 Definitions, Acronyms, and Abbreviations

- SRS – Software Requirements Specification
- LMS – Library Management System
- DB – Database
- FR – Functional Requirement
- NFR – Non-Functional Requirement
- PK / FK – Primary Key / Foreign Key

## 1.4 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications
- Python 3 official documentation ([docs.python.org](https://docs.python.org))
- SQLite documentation ([sqlite.org](https://sqlite.org))

## 1.5 Overview

The rest of this document is organised as follows: Section 2 gives an overall description of the product. Section 3 lists specific functional and non-functional requirements, along with the external interfaces and database schema. Section 4 describes the system using simple use-case and workflow models. Section 5 contains supporting appendices.

# 2. Overall Description

## 2.1 Product Perspective

The Library Management System is a standalone web application with a Python backend, a SQL database, and a browser-based frontend built with HTML and CSS. It does not depend on any external system. The

database is created from database/schema.sql and initial data — including the single Admin account — is loaded from database/seed\_data.sql during setup.

## 2.2 Product Functions

- Admin manages Librarian accounts (create, update, remove)
- Admin manages book stock across all sections, including minimum stock levels
- Librarians (multiple) register, view, update, and remove members
- Librarians search for books and issue a book to a member
- Librarians return a book and calculate a fine if it is overdue
- View the history of all transactions, including which librarian handled each one

## 2.3 User Characteristics

The system has two types of user: Admin and Librarian. There is no public or self-service registration for either role or for members — all accounts and member records are created internally.

| Role      | Responsibilities                                                                                                                                      |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Admin     | A single Admin account manages Librarian accounts and manages book stock (adding books, updating stock counts, monitoring minimum stock per section). |
| Librarian | Multiple Librarian accounts, created by the Admin, can each log in independently to register/manage members and to issue or return books.             |

Both Admin and Librarian users are assumed to have basic computer literacy but no programming knowledge, so the interface must use plain, numbered menu options rather than commands.

## 2.4 Constraints

- The application must run on a computer with Python 3 installed, inside a virtual environment (venv/).
- The database engine is a SQL database (e.g. MySQL); its schema is defined in database/schema.sql and database credentials are supplied through a .env file, never hardcoded or committed to source control.
- The frontend is limited to HTML and CSS pages served by the Python backend; no JavaScript framework is required.
- The system does not provide a public sign-up/registration screen for anyone. The Admin account is not created through the application at all — it is inserted directly into the database via database/seed\_data.sql. Only the Admin can create Librarian accounts, and only a Librarian can create member records.

## 2.5 Assumptions and Dependencies

- It is assumed that the Admin account is set up once, directly in the database through seed\_data.sql, before the application is first used — not through any login/registration form.
- It is assumed that multiple librarians may use the system, but not necessarily at the exact same moment on the same machine, since it is a single local application.
- It is assumed that the computer running the application has enough disk space to store the database file.
- The project depends on Python's database connector library for the chosen SQL database, and a lightweight Python web framework (e.g. Flask) to serve the HTML/CSS frontend pages.

## 3. Specific Requirements

### 3.1 Functional Requirements

#### 3.1.1 Admin Management

| ID     | Requirement                                                                                                                                                                                         |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-0.1 | The system shall not provide any application screen to create the Admin account; the single Admin row shall be inserted directly into the database by running database/seed_data.sql during setup.  |
| FR-0.2 | The system shall allow the pre-seeded Admin account to log in using its own credentials, kept separate from Librarian accounts.                                                                     |
| FR-0.3 | The system shall allow the Admin to create, update, and remove Librarian accounts through the application (unlike the Admin account itself, Librarian accounts are created at runtime, not seeded). |
| FR-0.4 | The system shall allow the Admin to view all Librarian accounts and their activity (books issued/returned handled by each).                                                                         |
| FR-0.5 | The system shall allow the Admin to add, update, and remove book stock across all sections/categories.                                                                                              |
| FR-0.6 | The system shall allow the Admin to view current stock count for every section and identify sections below the minimum stock level.                                                                 |
| FR-0.7 | The system shall not provide any public or self-service registration screen for Librarians or members; Librarian accounts are created only by the Admin, and members only by a Librarian.           |

#### 3.1.2 Book Management

| ID     | Requirement                                                                                                     |
|--------|-----------------------------------------------------------------------------------------------------------------|
| FR-1.1 | The system shall allow the Admin to add a new book with title, author, ISBN, section/category, and stock count. |
| FR-1.2 | The system shall allow the Admin to update details or stock count of an existing book.                          |

|        |                                                                                                                                                          |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-1.3 | The system shall allow the Admin to delete a book record from the database.                                                                              |
| FR-1.4 | The system shall allow a Librarian to search for a book by title, author, ISBN, or section.                                                              |
| FR-1.5 | The system shall display the availability status (Available / Issued) and current stock count of every book.                                             |
| FR-1.6 | The system shall maintain a minimum stock count of 5 books for every section; if a section's stock falls below 5, the system shall flag it as low stock. |

### 3.1.3 Member Management

| ID     | Requirement                                                                                                                                                         |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-2.1 | The system shall allow a Librarian to register a new member (customer) with name, member ID, and contact details. There is no self-registration option for members. |
| FR-2.2 | The system shall allow any Librarian to view the list of all registered members, regardless of which librarian registered them.                                     |
| FR-2.3 | The system shall allow a Librarian to update member contact details.                                                                                                |
| FR-2.4 | The system shall allow a Librarian to remove a member from the database.                                                                                            |

### 3.1.4 Transaction (Issue / Return) Management

| ID     | Requirement                                                                                                                     |
|--------|---------------------------------------------------------------------------------------------------------------------------------|
| FR-3.1 | The system shall allow any logged-in Librarian to issue a book to a member and record the issue date and the issuing librarian. |
| FR-3.2 | The system shall not allow a book to be issued if its section stock count is zero.                                              |
| FR-3.3 | The system shall allow any logged-in Librarian to record the return of a book and update its availability and stock count.      |
| FR-3.4 | The system shall calculate and display a fine if a book is returned after the due date.                                         |
| FR-3.5 | The system shall maintain a transaction history of every issue and return, including which librarian handled it.                |

## 3.2 Non-Functional Requirements

| ID    | Requirement                                                                                                                                    |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-1 | Performance: The system shall respond to any search or transaction request within 2 seconds.                                                   |
| NFR-2 | Reliability: The system shall not lose data in the event of an abnormal program exit; all changes are committed to the database immediately.   |
| NFR-3 | Usability: The menu-driven interface shall be simple enough for a librarian with no technical background to operate after a short walkthrough. |

|       |                                                                                                                                                |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-4 | Security: Only an authenticated librarian shall be able to add, update, or delete records.                                                     |
| NFR-5 | Maintainability: The code shall be organised into separate modules (book, member, transaction, database) so each can be updated independently. |
| NFR-6 | Portability: The system shall run on any machine with Python 3 installed, without needing a separate database server.                          |

### 3.3 External Interface Requirements

User Interface: A browser-based interface built with HTML and CSS. Pages are grouped by role — Admin sees librarian and stock management screens; Librarians see member and transaction screens — with plain forms and buttons rather than any command-line input.

Hardware Interface: Standard keyboard, mouse, and monitor for input/output; no special hardware is required.

Software Interface: Python 3 with a lightweight web framework (e.g. Flask) to serve the HTML/CSS pages in frontend/, and a database connector library in src/db\_connection.py to communicate with the SQL database defined in database/schema.sql.

Communication Interface: None required beyond the local browser-to-backend connection, as the system runs entirely on a single local machine.

### 3.4 Database Schema

The database consists of five tables, described below. schema.sql creates all five tables; seed\_data.sql inserts the Admin row and any sample/test data.

#### 3.4.1 Admins table

| Field    | Type         | Description                                       |
|----------|--------------|---------------------------------------------------|
| admin_id | INTEGER (PK) | Unique identifier for the admin, auto-incremented |
| username | TEXT         | Login username for the admin                      |
| password | TEXT         | Login password (stored as a hash)                 |

#### 3.4.2 Librarians table

| Field        | Type         | Description                                            |
|--------------|--------------|--------------------------------------------------------|
| librarian_id | INTEGER (PK) | Unique identifier for each librarian, auto-incremented |
| name         | TEXT         | Full name of the librarian                             |
| username     | TEXT         | Login username for the librarian                       |
| password     | TEXT         | Login password (stored as a hash)                      |

|                     |              |                                                               |
|---------------------|--------------|---------------------------------------------------------------|
| created_by_admin_id | INTEGER (FK) | References admin_id; records which admin created this account |
|---------------------|--------------|---------------------------------------------------------------|

### 3.4.3 Books table

| Field       | Type         | Description                                                             |
|-------------|--------------|-------------------------------------------------------------------------|
| book_id     | INTEGER (PK) | Unique identifier for each book, auto-incremented                       |
| title       | TEXT         | Title of the book                                                       |
| author      | TEXT         | Author of the book                                                      |
| isbn        | TEXT         | ISBN number of the book                                                 |
| section     | TEXT         | Section/category the book belongs to (e.g. Fiction, Science, Reference) |
| stock_count | INTEGER      | Number of copies currently in stock for this book                       |
| min_stock   | INTEGER      | Minimum stock required for this section (default 5)                     |
| status      | TEXT         | 'Available' or 'Issued', derived from stock_count                       |

### 3.4.4 Members table

| Field                      | Type         | Description                                                             |
|----------------------------|--------------|-------------------------------------------------------------------------|
| member_id                  | INTEGER (PK) | Unique identifier for each member, auto-incremented                     |
| name                       | TEXT         | Full name of the member                                                 |
| email                      | TEXT         | Contact email of the member                                             |
| registered_by_librarian_id | INTEGER (FK) | References librarian_id; records which librarian registered this member |

### 3.4.5 Transactions table

| Field          | Type         | Description                                                              |
|----------------|--------------|--------------------------------------------------------------------------|
| transaction_id | INTEGER (PK) | Unique identifier for each transaction, auto-incremented                 |
| book_id        | INTEGER (FK) | References book_id in the Books table                                    |
| member_id      | INTEGER (FK) | References member_id in the Members table                                |
| librarian_id   | INTEGER (FK) | References librarian_id; records which librarian handled the transaction |



|             |      |                                                       |
|-------------|------|-------------------------------------------------------|
| issue_date  | DATE | Date the book was issued                              |
| return_date | DATE | Date the book was returned (NULL if not yet returned) |

## 3.5 Frontend Pages

The frontend/ folder holds the HTML pages and a shared CSS stylesheet. Each page maps to one or more functional requirements above.

| Page                     | Purpose                                                                                                                  |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------|
| login.html               | Single login form used by both Admin and Librarian; backend checks credentials against the Admins or Librarians table.   |
| admin_dashboard.html     | Admin-only page listing librarian accounts and section-wise stock, with forms to add/update/remove librarians and stock. |
| librarian_dashboard.html | Librarian-only page with links to manage members and to issue/return books.                                              |
| books.html               | Search/view books by title, author, ISBN, or section, showing availability and stock count.                              |
| members.html             | Register a new member, and view/update/remove existing members.                                                          |
| issue_return.html        | Form to issue a book to a member, or record a return and show any fine due.                                              |
| static/css/style.css     | Shared stylesheet giving all pages a consistent layout, colours, and form styling.                                       |

## 4. System Models

### 4.1 Use Case Summary

| Use Case        | Description                                                                                                                    |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------|
| Add Book        | Librarian enters book details; system stores a new record in the Books table.                                                  |
| Search Book     | User enters a title, author, or ISBN; system displays matching books and their availability.                                   |
| Register Member | Librarian enters member details; system stores a new record in the Members table.                                              |
| Issue Book      | Librarian selects a book and a member; system checks availability, creates a transaction, and marks the book as issued.        |
| Return Book     | Librarian selects an active transaction; system records the return date, calculates any fine, and marks the book as available. |

## 4.2 Process Flow (Issue Book)

- Step 1: Librarian selects 'Issue Book' from the main menu.
- Step 2: System asks for the book ID and member ID.
- Step 3: System checks the book's status in the database.
- Step 4: If the book is available, the system creates a new transaction record and updates the book status to 'Issued'.
- Step 5: If the book is not available, the system displays a message and returns to the main menu.

## 4.3 Project Folder Structure

The project is organised the same way it will be built in VS Code: SQL files under database/, Python backend files under src/, and the HTML/CSS frontend under frontend/.

```
library_management_system/
|
+-- venv/                                # virtual environment (not in git)
|
+-- database/
|   +-- schema.sql                      # CREATE DATABASE / CREATE TABLE statements
|   +-- seed_data.sql                  # inserts the Admin account + sample data
|
+-- src/
|   +-- __init__.py                    # empty, marks folder as a Python package
|   +-- db_connection.py               # connection logic ONLY
|   +-- auth.py                       # login check for Admin and Librarian
|   +-- admin_repository.py           # manage librarians, manage stock
|   +-- librarian_repository.py       # manage members
|   +-- book_repository.py            # book CRUD, section/stock logic
|   +-- transaction_repository.py     # issue, return, fine calculation
|   +-- main.py                       # entry point, imports & runs app
|
+-- frontend/
|   +-- templates/
|   |   +-- login.html
|   |   +-- admin_dashboard.html
|   |   +-- librarian_dashboard.html
|   |   +-- books.html
|   |   +-- members.html
|   |   +-- issue_return.html
|   +-- static/
|   |   +-- css/
|   |   |   +-- style.css
|
+-- .env                               # DB credentials (never hardcode/commit)
+-- .gitignore                         # ignores venv/, .env, __pycache__
+-- requirements.txt                   # pip freeze > requirements.txt
+-- README.md
```

database/ holds only SQL files. src/ holds only Python backend files, split by responsibility — admin\_repository.py contains Admin-only functions, librarian\_repository.py contains Librarian-only functions, and book\_repository.py / transaction\_repository.py are shared by both roles. frontend/ holds only HTML and CSS, kept separate from the Python code.

## 5. Appendices

### 5.1 Glossary

- Book: A physical item in the library catalogue that can be issued to a member.
- Member: A registered user of the library who can borrow books.
- Transaction: A record linking a book, a member, and the issue/return dates.
- Fine: A charge applied when a book is returned after its due date.

### 5.2 Future Enhancements

- Add JavaScript for client-side validation and a more interactive frontend experience.
- Add password hashing and session-based login security for Admin and Librarian accounts.
- Add email or SMS reminders for due dates.